

torch.compiler.set_stance#

https://pytorch.org/docs/stable/generated/torch.compiler.set_stance.html

Description:

Set the current stance of the compiler. Can be used as a function, context manager, or decorator. Do not use this function inside a `torch.compile` region - an error will be raised otherwise. The stance to set the compiler to. Valid values are: "default": The default stance, used for normal compilation. "force_eager": Ignore all `torch.compile` directives.

Function Signature

```
torch.compiler.set_stance(stance='default', *,  
skip_guard_eval_unsafe=False, force_backend=None)[source]#
```

Parameters

Code Examples

```
@torch.compile
def foo(x): ...

@torch.compiler.set_stance("force_eager")
def bar():
    # will not be compiled
    foo(...)

bar()

with torch.compiler.set_stance("force_eager"):
    # will also not be compiled
    foo(...)

torch.compiler.set_stance("force_eager")
# will also not be compiled
foo(...)
torch.compiler.set_stance("default")

# will be compiled
foo(...)
```

Detailed Documentation

Documentation

Set the current stance of the compiler. Can be used as a function, context manager, or

decorator. Do not use this function inside a `torch.compile` region - an error will be raised otherwise.

The stance to set the compiler to. Valid values are:

"default": The default stance, used for normal compilation.

"force_eager": Ignore all `torch.compile` directives.

"eager_on_recompile": Run code eagerly when a recompile is necessary. If there is cached compiled code valid for the input, it will still be used.

"fail_on_recompile": Raise an error when recompiling a function.

"eager_then_compile": Run the first invocation in eager mode, then compile on subsequent calls. This is beneficial for dynamic shapes as it allows inferring dynamism from the first two invocations instead of wasting a static compile on the first invocation.

"aot_eager_then_compile": Run the first invocation with AOT eager to get memory benefits from activation checkpointing, then compile on subsequent calls. Like `eager_then_compile`, this improves handling of dynamic shapes by avoiding an initial static compile.

`skip_guard_eval_unsafe` (bool) –

A flag to run only differentiating guards. CAUTION - This flag is unsafe and should only be used if your setup meets the following conditions.

`torch.compile` uses a guard system to support recompilations and choose which compiled artifact to run at runtime. These guards, though efficient, add some overhead, which may impact performance in scenarios where you need to optimize for minimal guard processing time. This API enables you to disable guard evaluation, assuming that

you have warmed up the compiled model with a sufficient variety of inputs. This assumption means that, after the warmup phase, no further recompilations will be necessary. If this assumption fails, there is a risk of silently producing incorrect results (hence the term “unsafe” in the API name).

`force_backend` (Optional[Union[str, Callable[[...], Any]]]) – If `device` is “default”, this argument can be used to force `torch.compile` to use a specific backend. Otherwise, an error is raised.